
Redyx — Spam Service Test Cases

idityaGE

2026-05-10

Contents

Spam Service — Test Case Specification **2**

 Test Infrastructure 2

 1. Blocklist — LoadBlocklist Test 2

 2. Blocklist — CheckKeywords Tests 3

 3. Blocklist — CheckURLs Tests 3

 4. Blocklist — ExtractURLs Tests 3

 5. Dedup Checker — DedupChecker . Check Tests 4

 6. Server — CheckContent Tests 5

 Patterns & Conventions 6

Spam Service — Test Case Specification

Project Name	Redyx
Module Name	Spam
File	internal/spam/spam_test.go
Total Tests	22
Package	spam (same-package test)
Created By	idityaGE
Date of Creation	2026-05-10

Test Infrastructure

Component	Implementation
Logger	<code>zap.NewNop()</code>
Redis	<code>miniredis.RunT(t)</code> (in-process, via <code>testRedis</code> helper)
Blocklist File	<code>data/blocklist.json</code> loaded via <code>LoadBlocklist</code>
Server Helper	<code>testServer(t)</code> — composes <code>testBlocklist</code> + <code>testRedis</code> + <code>NewDedupChecker</code> + <code>NewServer</code>
Helper convention	<code>t.Helper()</code> in all setup helpers
Skip guard	<code>t.Skipf</code> if <code>data/blocklist.json</code> not found on disk

1. Blocklist — LoadBlocklist Test

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_SPAM_001	<code>TestLoadBlocklist</code>	<code>data/blocklist.json</code> exists on disk	<code>Call <code>LoadBlocklist("data/blocklist.json")</code></code>	<code>path = "data/blocklist.json"</code>	Returns non-nil <code>Blocklist</code> ; <code>keywords</code> list non-empty; <code>domains</code> list non-empty	PASS

2. Blocklist — CheckKeywords Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_SPAM_002	TestCheckKeywords_Clean	Blocklist loaded	Call <code>bl.CheckKeywords(text)</code> (text is a normal post about programming")	content = clean	matched=false	PASS
TC_SPAM_003	TestCheckKeywords_Match	Blocklist loaded	Call <code>bl.CheckKeywords("buy now everyone, BUY NOW before it's too late!")</code>	content contains keyword	matched=true	PASS
TC_SPAM_004	TestCheckKeywords_CaseInsensitive	Blocklist loaded	Call <code>bl.CheckKeywords("CLICK HERE for amazing deals")</code>	content contains keyword "click here" in uppercase	matched=true (case-insensitive matching)	PASS

3. Blocklist — CheckURLs Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_SPAM_005	TestCheckURLs_Clean	Blocklist loaded	Call <code>bl.CheckURLs(["https://golang.org", "https://github.com"])</code>	urls = legitimate domains	matched=false	PASS
TC_SPAM_006	TestCheckURLs_BlockedDomain	Blocklist loaded	Call <code>bl.CheckURLs(["I blocked domain site.com/payload"])</code>	urls = known blocked domain	matched=true	PASS
TC_SPAM_007	TestCheckURLs_BitLy	Blocklist loaded	Call <code>bl.CheckURLs(["https://bit.ly/123456"])</code>	urls = bit.ly shortened URL (URL host is blocked)	matched=true	PASS

4. Blocklist — ExtractURLs Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_SPAM_008	TestExtractURLs — BareURLs		Call ExtractURLs("Check out https://example.com and http://test.org/page")	content with 2 bare URLs	Returns slice of length 2	PASS
TC_SPAM_009	TestExtractURL — MarkdownLinks		Call ExtractURLs("Visit [my site](https://example.com/for more)")	content with 1 markdown link URL	Returns slice of length ≥ 1	PASS
TC_SPAM_010	TestExtractURLs — NoURLs		Call ExtractURLs("This is a plain text post with no links")	content with no URLs	Returns slice of length 0	PASS

5. Dedup Checker — DedupChecker . Check Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_SPAM_011	TestCheckDuplicate FirstTime	miniredis running; fresh dedup store	Call dc.Check(ctx, "user1", "Hello world")	userId = "user1", content = "Hello world"	isDup=false, hash is non-empty string	PASS
TC_SPAM_012	TestCheckDuplicate SecondTime	miniredis running; same content already submitted by user1	Submit same content twice for same user	userId = "user1", content = "Hello world" (twice)	1st call: isDup=false; 2nd call: isDup=true	PASS
TC_SPAM_013	TestCheckDuplicate DifferentUser	miniredis running; user1 already submitted content	Submit same content for user2	userId = "user2", content = "Hello world"	isDup=false (dedup is per-user scoped)	PASS
TC_SPAM_014	TestCheckDuplicate NormalizesContent	miniredis running; fresh dedup store	Submit " Hello World " then "hello world" for same user	user1 with whitespace-variant content	isDup=true; both calls return same hash (content normalized before hashing)	PASS

6. Server – CheckContent Tests

Test Case ID	Test Name	Pre-Condition	Test Steps	Test Data	Expected Result	Status
TC_SPAM_015	TestCheckContent_Clean	Server initialized with blocklist + dedup	Call <code>srv.CheckContent</code> with clean content	<code>userId = "user1",</code> <code>contentType = "post_body",</code> <code>content = "This is a normal discussion about Go programming"</code>	<code>result=SPAM_CHECK_RESULT_CLEAN,</code> <code>reasons=[]</code> (empty)	PASS
TC_SPAM_016	TestCheckContent_BlockedKeyword	Server initialized	Call <code>srv.CheckContent</code> with content containing blocked keyword	<code>content = "Hey! Buy now before it's too late!"</code>	<code>result=SPAM_CHECK_RESULT_SPAM,</code> <code>len(reasons)>0</code>	PASS
TC_SPAM_017	TestCheckContent_BlockedURL	Server initialized	Call <code>srv.CheckContent</code> with blocked URL in <code>urls</code> field	<code>urls = ["https://malicious-site.com/payment"]</code>	<code>result=SPAM_CHECK_RESULT_SPAM</code>	PASS
TC_SPAM_018	TestCheckContent_BlockedURL_InContent	Server initialized	Call <code>srv.CheckContent</code> with blocked URL embedded in content text	<code>content = "Visit https://bit.ly/1234 for free stuff"</code>	<code>result=SPAM_CHECK_RESULT_SPAM</code> (URL extracted from content text)	PASS
TC_SPAM_019	TestCheckContent_Duplicate	Server initialized	Submit same request twice	<code>userId = "user1",</code> <code>content = "Some unique post content here"</code>	1st resp: <code>isDuplicate=false,</code> con- tentHash≠""; 2nd resp: <code>isDuplicate=true</code>	PASS
TC_SPAM_020	TestCheckContent_VagueReasons	Server initialized	Submit content with multiple spam keywords; inspect reasons	<code>content = "Buy now and get free money!"</code>	reasons must NOT contain exact keyword strings like "buy now" or "free money" (reasons are deliberately vague for privacy)	PASS

Patterns & Conventions

Pattern	Implementation
Package scope	<code>package spam</code> (white-box / same-package)
Logger	<code>zap.NewNop()</code> (silent)
In-process Redis	<code>miniredis.RunT(t)</code> — auto-cleaned on <code>t.Cleanup</code>
Blocklist skip guard	<code>t.Skipf</code> when <code>data/blocklist.json</code> not present — safe in CI without data file
Content normalization	Dedup normalizes whitespace + lowercases before hashing (SHA-based)
Reason vagueness	Spam reasons returned to clients are deliberately obfuscated (no exact keyword exposure)
Helper tagging	<code>t.Helper()</code> in <code>testBlocklist</code> , <code>testRedis</code> , <code>testServer</code>
Reference pattern	This file is the canonical pattern for all other service test files in the codebase